ZENFLOW ARCHITECTURE GUIDE

How the website works and how the frontend and backend fit together

This document explains the current Zenflow codebase as implemented in the repository, with emphasis on the React frontend, the Express backend, the shared data model, and the real request flows between them.

Repository: `zenflow_app` | Generated: March 13, 2026 | Audience: product, engineering, onboarding

PUBLIC SITE

What visitors see before login

Zenflow exposes a marketing landing page, public legal pages, a contact/FAQ section, and blog articles that search engines can crawl and users can read without creating an account.

AUTHENTICATED APP

What users get after login

After authentication, the app switches into a personal workspace with dashboard metrics, focus timer, meditation, Sudoku, brain games, planner, notes, streaks, and profile settings.

SINGLE DEPLOYMENT

How it is hosted

In production, one Node service serves the built frontend and also handles the API under `/api`. That same deployment is what the website and the app use.

HIGH-LEVEL RUNTIME

One app shell, one API, one shared data model

Browser or Android WebView

- Loads the React single-page app
- Stores session locally

->

Frontend app

- `client/index.html`
- `client/src/main.tsx`
- `client/src/App.tsx`

->

Express backend

- JWT auth and account APIs
- Logs, meta, leaderboard, email flows

- Calls API through `apiUrl()`

- Feature components and shared utils

- MongoDB first, file fallback in local dev

Production model: Express serves `client/dist` and routes all unknown URLs back to `index.html`, so React owns the screen-level navigation while Express owns the API and deployment shell.

Development model: Vite runs on `localhost:5173`, Express runs on `localhost:4100`, and the Vite dev server proxies `/api` to the backend.

FRONTEND ARCHITECTURE

How the React side is organized

1. Entry layer

- `client/index.html` provides the root div, base SEO metadata, GA4 tag, and fonts.
- `client/src/main.tsx` bootstraps React and initializes analytics.

2. App shell

- `client/src/App.tsx` is the control center.
- It restores the session from local or session storage.
- It decides whether the user is seeing a public page, the login overlay, or an authenticated tool/dashboard view.
- It updates canonical tags and social metadata for public pages.
- It manually sends page-view analytics for the SPA.

3. Feature components

- Public: `MarketingLanding.tsx`, `BlogPages.tsx`, `StaticPages.tsx`
- Authenticated: `Dashboard.tsx`, `PomodoroTimer.tsx`, `MeditationTimer.tsx`, `SudokuTrainer.tsx`, `BrainArcade.tsx`, `PlannerBoard.tsx`, `ProfileCenter.tsx`

4. Shared utilities

- `utils/api.ts` chooses the correct API base URL for web and Android.
- `utils/analytics.ts` wraps GA4 page-view and login tracking.
- `utils/profile.ts` defines the flexible profile meta shape.
- `utils/wellness.ts` turns logs into points, quests, streaks, and rewards.
- `utils/planner.ts` shapes planner data and notifications.

5. Client-side navigation model

Zenflow does not use React Router. Instead, `App.tsx` resolves public URLs itself and uses a local `selected` state to switch between internal app screens such as dashboard, planner, timer, Sudoku, and profile.

> This means the app behaves like a custom SPA shell: public pages map to real paths for SEO, while in-app tool views are controlled by React state after login.

WEBSITE BEHAVIOR

What the user journey looks like

Discover

Visitors land on the public homepage, blog, FAQ, about, contact, privacy, terms, and cookie pages.

Authenticate

Login and registration open in an overlay. Sessions can be remembered in local storage or kept for the session only.

Use tools

Authenticated users move between dashboard, focus timer, meditation, Sudoku, brain games, planner, and profile.

Persist progress

Timers, notes, planner entries, best scores, streak data, and preferences are saved through the backend and reloaded on return.

Public section

The landing page is designed to persuade visitors to try the product. Public blog articles and legal pages are available without login. Metadata is rewritten by the app shell so these URLs have proper titles, descriptions, canonicals, and analytics.

Private section

Once a valid session exists, the same SPA changes into a user dashboard. The bottom navigation and top app shell switch the view by updating React state instead of navigating to separate server-rendered pages.

BACKEND ARCHITECTURE

How the Express server is structured

Runtime and middleware

- `server/index.js` contains the server entry point and almost all backend logic.
- Express is configured with `cors()` and JSON request parsing.
- `/health` reports basic service and storage status.

Persistence model

- Primary storage is MongoDB via Mongoose.
- Collections/models: `User`, `Log`, and `Meta`.
- If MongoDB is unavailable in local development, the backend can fall back to `server/data.json`.
- In production, MongoDB is required and startup exits if it is missing.

Security and auth

- JWT tokens are signed with `ZENFLOW_SECRET` and expire after 7 days.
- `authMiddleware` protects account, logs, meta, and leaderboard endpoints.
- Passwords are hashed with `bcryptjs`.
- Login attempts are throttled to reduce brute-force retries.

Optional integrations

- Google sign-in can be enabled with `GOOGLE_CLIENT_ID`.
- Password reset and email verification can send through SMTP and/or Resend.
- There is also an admin endpoint to broadcast Android release announcements.

DATA MODEL

What gets saved where

STORE	PURPOSE	EXAMPLES
`User`	Identity and account state	Username, full name, email, password hash, Google identity, verification status, created date, login count
`Log`	Time-series activity records used for progress and rewards	Pomodoro minutes, meditation minutes, Sudoku completions, memory sessions, reaction sessions, steps
`Meta`	Flexible per-user JSON blob for richer app state	Theme, profile fields, journals, planner data, todos, Sudoku best times, reaction best time, other feature state

Practical split: logs hold measurable activity over time, while meta holds personalized structured state that does not fit cleanly into a tiny numeric event record.

Example: completing a Pomodoro session usually writes a log entry; editing daily notes or saving planner tasks usually writes to meta.

FRONTEND AND BACKEND INTEGRATION

The main request flows

USE CASE	FRONTEND BEHAVIOR	BACKEND BEHAVIOR
Session restore	`App.tsx` reads the stored token, then calls `GET /api/me`.	Validates the JWT and returns the normalized account payload used across the app.
Register	`Login.tsx` posts account details to `POST /api/register`.	Validates fields, hashes the password, stores the user, and optionally sends an email verification code.
Login	`Login.tsx` posts username/email and password to `POST /api/login`.	Checks credentials, enforces throttle rules, updates login stats, and returns a JWT plus account data.
Dashboard load	`Dashboard.tsx` requests `/api/logs`, `/api/meta`, and `/api/leaderboard/trends` in parallel.	Returns user activity, saved metadata, and the top-user trend summary used for charts.
Save progress	Feature components send `POST /api/logs` for activity and `POST /api/meta` for saved state.	Upserts records keyed by user and merges meta so feature-specific state persists across sessions.
Leaderboards	Dashboard, Sudoku, and Brain Arcade request leaderboard endpoints.	Backend aggregates logs and meta into point tables, trend graphs, Sudoku best times, and reaction-time rankings.

Example flow: login and dashboard

```
Login.tsx -> POST /api/login -> JWT + account -> App.tsx stores session -> GET /api/me -> Dashboard.tsx GET /api/logs + /api/meta + /api/leaderboard/trends
```

Example flow: Sudoku best time

```
SudokuTrainer.tsx -> POST /api/logs (completion) -> POST /api/meta (bestTimesMs) -> GET /api/leaderboard/game-records -> top users shown in UI
```

ENVIRONMENT AND DEPLOYMENT

How local development, production, and Android fit together

Local development

- Frontend: `npm run dev:client` starts Vite.
- Backend: `npm run dev:server` starts Express.
- Vite proxies `/api` to `http://localhost:4100`.
- If port 4100 is occupied, the backend retries the next port.

Production on Render

- `render.yaml` defines one Node web service.
- `npm run render:build` installs dependencies and builds the client.
- `npm run render:start` starts the server, which also serves the built frontend.
- Required environment variables are primarily `MONGODB_URI` and `ZENFLOW_SECRET`.

API base resolution

On the web, `apiUrl()` prefers same-origin relative paths, which keeps deployment simple. On Android, it can switch to an explicit host such as `10.0.2.2:4100` for the emulator or `https://zenflow.bio` in production.

Capacitor Android wrapper

The frontend is also packaged as an Android app through Capacitor. The same React bundle is used, but the networking layer can point at a mobile-friendly backend host when needed.

KEY FILES

The shortest useful map of the codebase

- **`client/index.html`**: base HTML shell, GA4 tag, root div, SEO defaults
- **`client/src/main.tsx`**: React bootstrap and analytics init
- **`client/src/App.tsx`**: session restore, public-page routing, app-shell view switching, metadata, analytics
- **`client/src/components/Login.tsx`**: login, registration, verification, reset, Google sign-in UI
- **`client/src/components/Dashboard.tsx`**: charts, notes, progress summary, leaderboards
- **`client/src/components/SudokuTrainer.tsx`**: Sudoku play state, best-time persistence, leaderboard reads
- **`client/src/utils/api.ts`**: same-origin web API logic and Android host selection
- **`client/src/utils/profile.ts`**: profile meta schema used across dashboard, planner, games, and notes
- **`server/index.js`**: Express entry, auth, storage, APIs, email flows, static serving
- **`render.yaml`**: production build/start definition for Render

IMPORTANT OBSERVATIONS

Things to know if you are extending this app

- The codebase currently centralizes nearly all backend logic in one file, `server/index.js`. That is workable now, but it will become the first scaling pressure point as features grow.
- The frontend uses a custom state-driven navigation model rather than React Router. That keeps things simple, but route logic and app-view logic both live in `App.tsx`.
- The flexible `meta` document makes it easy to ship new features quickly because new per-user fields can be added without a new collection.
- The public site and the logged-in app are tightly integrated. This is convenient for deployment and same-origin API calls, but SEO work has to be handled carefully because the UI is still a SPA.
- The contact page frontend posts to `/api/contact`, but I did not find a matching backend route in the current server code. If that form is expected to work, it likely still needs a server implementation.

> **Short version:** Zenflow is a single-page React app backed by an Express API. The frontend owns the user experience, the backend owns identity and persistence, and

> the bridge between them is a small set of JWT-protected JSON endpoints under `/api`.

This guide was generated from the current repository structure and implementation details present in `client/`, `server/`, and `render.yaml`.